

LoRA / QLoRA Implementation Plan

Project: Domain adaptation of an open LLM on the Astro_book Jyotisha corpus **Revised:** 2026-08-26 (rev 5 — validated; results in §13, §14, §15) **Status:**  **Complete.** Both arms trained, comparison measured.

Headline result: QLoRA cut peak VRAM by **54.7%** (12.22 → 5.54 GB) for a 14.5% step-time cost and an eval-loss difference of **0.0107** — inside noise. 3 of 5 pre-registered predictions held; the two misses are analysed in §13 and are the most instructive part of the run.

Phase 6 (§14): at 8B the trade becomes decisive — QLoRA is **37× faster** than bf16 on this card, because bf16 silently pages ~11 GB to system RAM instead of failing cleanly.

Validation (§15): against the untuned base on held-out books, the adapter cuts perplexity **15.96 → 4.25 (3.8×)** and eliminates markdown preamble (65% → 0%). Factual accuracy remains unmeasured.

Rev 2 supersedes the free-tier Kaggle plan. An RTX 5070 Ti was found on this machine, which removes the T4, the 12-hour session cap, the fp16-only constraint, and the API spend. Numbers below marked *measured* were taken on this hardware; numbers marked *projected* have not yet been observed.

1. Objective

Primary — implement and understand LoRA and QLoRA. The deliverable is a working implementation of both, run end-to-end on real data, with a measured comparison between them.

Secondary — a domain-adapted Jyotisha Q&A adapter. A by-product. If it turns out mediocre, the project still succeeded.

Definition of done: you can explain, from your own measurements rather than from documentation, what quantizing the frozen base to 4 bits buys and costs.

Non-goals for v1: no retrieval, no serving, no UI, no production tutor.

What this adapter will and will not do

LoRA installs **style, idiom, and reasoning shape**. Trained on this corpus the model will reason in houses / lords / dashas / yogas and structure answers the way a Jyotisha text does.

LoRA does **not** reliably install **facts** at this corpus size. Asked "what does Phaladeepika say about Saturn in the 7th," it will produce fluent, correctly-shaped, confidently-wrong answers. That is an accepted limitation of v1, not a defect to debug. The fix is retrieval (§11), not more training.

2. Hardware and environment (*measured*)

GPU	NVIDIA GeForce RTX 5070 Ti — 16,303 MiB
Compute capability	12.0 / sm_120 (Blackwell)
Driver / CUDA	591.86 / CUDA 13.1
CPU · RAM · Disk	Ryzen 9 9900X (12C/24T) · 61.6 GB · 1.47 TB free
Python	3.14.7
torch	2.11.0+cu128 — sm_120 present in compiled arch list
bitsandbytes · peft · trl · transformers	0.50.1 · 0.20.0 · 1.10.0 · 5.15.1

`pipeline/check_env.py` passes every check, including a real NF4 forward pass and a LoRA backward pass. QLoRA and LoRA are both viable here.

Two consequences of Blackwell being new:

- A torch wheel built only to sm_90 imports fine, reports `cuda.is_available() == True`, then dies at the first real kernel. `check_env.py` therefore runs actual kernels rather than trusting imports. Re-run it after any torch change.
- `triton` is not installed, so Unsloth cannot run on this box. That only affects the optional Phase 6; the peft + bitsandbytes path needs no triton.

bf16 is available (it was not on the T4), so the fp16-only workarounds in the original plan are void.

3. Corpus (*measured*)

52 PDFs, 211 MB, ~7,400 pages of Jyotisha literature — classical Sanskrit translations, modern textbooks, technique monographs, prashna, astronomy, and two software manuals.

44 files usable → 1,904 chunks, ~2.97M tokens. Entirely English and romanized transliteration (zero Devanagari), so no tokenizer extension or multilingual base is required.

8 files excluded:

File(s)	Reason
ashtaka-varga , greatness-gayatri-jyotish , astrology-and-stock-market-forecasting , jyotisha-siddhanta-sara	Pure image scans, zero extractable text
lal-kitab-vol-1/2/3-1952	Scans; only extractable text is a distributor watermark
lal-kitab-1941	Real text layer, but legacy non-Unicode Hindi font → mojibake

The Lal Kitab tradition is therefore **entirely absent from v1**.

Chunk health after two extraction fixes

Check	Before	After
Pinned at size cap (sliced mid-sentence)	54%	0.2%
Starting mid-sentence	8.3%	0.9%
Private-use-area garbage glyphs	7.0% (2,136 in crux-of-astrology)	0%

Fix 1: the verse-marker regex was matching *inside* running prose; units starting lowercase are now merged back into their predecessor. Fix 2: PUA codepoints from broken subsetted fonts are stripped in `clean()` .

Tables flattened into line-soup remain and were left deliberately — the generation prompt has a `grounded: false` escape for them.

4. Methodology decisions

Spec-Driven Development — rejected

SDD resolves **intent ambiguity**. This project has none: LoRA/QLoRA training has a canonical shape. Its uncertainty is **empirical** — what VRAM 4-bit actually uses, whether loss flattens by epoch 2. No spec answers those. The forcing-function benefit was collected conversationally while scoping the work.

Test-Driven Development — adopted, scoped

ML pipeline bugs are silent. A chunking bug does not throw; it yields slightly worse data → a slightly worse adapter → a mediocre score misattributed to hyperparameters.

This has already paid for itself twice:

1. The 54%-pinned chunking bug produced perfectly valid-looking JSONL. Caught by a histogram, now guarded by an assertion.
2. `test_inject_and_count` failed at 6,208 trainable vs 2,048 expected. The gap was exactly one unwrapped `Linear` + bias: `inject_lora` froze only the layers it *wrapped*, leaving every untargeted `Linear`, `LayerNorm`

and embedding trainable — a **partial full fine-tune wearing a LoRA costume**. It would have trained fine and converged fine. Not findable by inspection.

Pre-registered predictions — adopted

Locked in \$5 before any GPU run, and checked mechanically by `06_compare.py` so the outcome cannot be rationalized after the fact.

5. Pre-registered predictions

Same base model, same data, same hyperparameters. **Only the quantization of the frozen base differs.**

Metric	LoRA (bf16)	QLoRA (4-bit NF4)	Prediction	Pass threshold
Peak VRAM	~12-13 GB	~6 GB	QLoRA uses 50-60% less	✓ 54.7%
Sec / step	baseline	slower	QLoRA 25-40% slower	✗ 14.5%
Final eval loss	baseline	comparable	within 0.15	✓ 0.0107
Adapter size on disk	—	—	identical (control)	✗ 50% (dtype)
Trainable params	—	—	identical (control)	✓ 0 diff

Falsification conditions. If any of these hold, "QLoRA is a VRAM-for-speed trade at negligible quality cost" is wrong for this setup and must be investigated, not explained away:

- QLoRA eval loss worse by > 0.15
- VRAM saving < 30%
- Adapter sizes or trainable counts **differ** — implies the LoRA config diverged between runs, i.e. a bug that invalidates the comparison
- Either run fails to beat the untuned base in stage 5 — implies bad **data**, not bad quantization

Why the same base model matters. Comparing 4B-QLoRA against 1.7B-LoRA changes two variables and teaches nothing about quantization. Qwen3-4B is chosen because it should fit *both* ways in 16 GB. If bf16 OOMs, fall back to Qwen3-1.7B **for both arms**, and record the OOM — it is itself a result.

6. Execution phases

Phase 0 — LoRA from scratch ✓ complete

`LoRALinear` : $y = Wx + (BA \cdot x)(\alpha/r)$, A Kaiming-init, B zero-init. **9/9 tests pass**, including bit-identical equivalence with `peft` (`atol=1e-6`) on shared weights — the test that distinguishes understanding from something-that-trains.

```
r=8    trainable  110,592 / 7,195,392  = 1.54%
r=32   trainable  442,368 / 7,527,168  = 5.88%
identity at init: True    <- B=0 makes the adapter an exact no-op
```

Phase 1 — Extract complete

```
python pipeline/01_extract.py --report
```

→ build/chunks.jsonl , 1,904 chunks. **Gate:** pinned at hard cap < 5%.  0.2%

Phase 2 — Generate pairs running

Local, free, on the 5070 Ti via Ollama. Supersedes the Claude Batch API plan (02_generate.py is retained and shares the same prompt via config.py).

```
python pipeline/02_generate_local.py --model qwen3:14b --workers 3
python pipeline/02_generate_local.py --finalize
```

Teacher choice is settled by measurement, not preference:

Model	tok/s	1 chunk (8 pairs)	Valid pairs
qwen2.5:7b	138	17.7 s (hit token cap)	0
qwen3:14b	75	14.6 s	8

The 7B rambles, never closes its JSON, and returns nothing usable. The 14B is half the raw tok/s but faster in practice because it is concise and terminates.

Do not enable request parallelism. OLLAMA_NUM_PARALLEL=4 *reduced* throughput from 4.2 to 1.1 chunks/min — at 97% VRAM the KV-cache slots thrash. =2 was also worse. Ollama's default is correct here.

Sustained rate: ~4.0-4.3 chunks/min → ~8 hrs for the full corpus. Resumable; appends per chunk and skips completed work.

Phase 3 — Split

```
python pipeline/03_split.py
```

Splits **by source book**, not by row — books share vocabulary and translator idiom, and a row split leaks that across the boundary.

Phase 4 — The A/B (*the core*)

```
python pipeline/04_train_hf.py --max-steps 20 --out runs/smoke
python pipeline/04_train_hf.py --no-qlora --out runs/lora-bf16
python pipeline/04_train_hf.py --qlora    --out runs/qlora-nf4
```

The entire difference between the two arms:

```
BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                    bnb_4bit_use_double_quant=True,
                    bnb_4bit_compute_dtype=torch.bfloat16)
```

Everything after that point is byte-identical. Use `04_train_hf.py` (plain peft + bitsandbytes), **not** the Unsloth script — Unsloth hides the quantization config, checkpointing patch and attention swap behind one call, and needs triton which is absent here.

Phase 5 — Measure and evaluate

```
python pipeline/06_compare.py runs/lora-bf16 runs/qlora-nf4
python pipeline/05_eval.py generate --adapter runs/qlora-nf4 --n 150
python pipeline/05_eval.py judge          # needs an API key
```

`05_eval.py judge` grades **faithfulness, not correctness** — no ground truth exists for "correct Jyotisha", but every test pair carries the `chunk_id` it came from, so the source passage is a real reference. Position-randomized A/B against the untuned base.

Phase 6 — 8B demonstration complete (see §14)

Ran both ways on Qwen3-8B. bf16 did **not** OOM as predicted — it oversubscribed VRAM and ran 37× slower. 4-bit fits in 6.17 GB and trains normally.

7. Test plan

Test	Asserts	Status
<code>test_lora_module.py</code>	own <code>LoRALinear</code> $\equiv$ <code>peft</code> ; <code>B=0</code> $\rightarrow$ identity; only A/B trainable; merge preserves function	 9/9
<code>test_eval_unswap.py</code>	A/B attribution correct under both flip values; includes a deliberately broken variant to prove the test has teeth	 5/5
<code>test_data_pipeline.py</code>	chunk caps, mid-sentence starts, PUA glyphs, <code>custom_id</code> injectivity, QC gates, book-level split leakage , chat format	 12/12
<code>test_train_smoke.py</code>	overfit 8 examples $\rightarrow$ loss collapses	to build

26 tests passing. `test_split_has_no_book_leakage` confirms zero train/test book overlap, so the eval numbers in §13 are trustworthy. Only `test_train_smoke.py` (overfit 8 examples) remains unbuilt.

8. Hyperparameter reference

Setting	Value	Rationale
Base (A/B)	Qwen3-4B-Instruct	Should fit 16 GB both ways — required for a valid comparison
Quantization	NF4 + double quant, bf16 compute	The QLoRA recipe. Purely a VRAM decision
Rank <code>r</code>	32	Not the common 8 — teaching a domain idiom, not a response format
<code>lora_alpha</code>	64	<code>alpha = 2r</code> , standard pairing
Target modules	all 7 linear	Matters more than rank . q/v-only leaves the MLP untouched
Seq length	1024	Pairs average ~500 tok
LR / schedule	1e-4, cosine, 20 warmup steps	~10× full-FT LR because only the adapter moves
Effective batch	16 (2 × 8 accum)	
Epochs	2	Overfits past 3 at this size. Tell: verbatim corpus recitation
Loss	<code>assistant_only_loss=True</code>	Otherwise capacity goes into reproducing the prompt
Optimizer	<code>paged_adamw_8bit</code> (QLoRA) / <code>adamw_torch</code> (LoRA)	

API renames verified by introspection (trl 1.10 / transformers 5.15) — older tutorials will crash:

`max_seq_length` → `max_length` , `evaluation_strategy` → `eval_strategy` , `warmup_ratio` → **gone**
(`warmup_steps`), `tokenizer=` → `processing_class=` , `torch_dtype=` → `dtype=` , `group_by_length` → **gone**.

9. Failure modes and responses

Symptom	Cause	Action
Generation collapses to <1 chunk/min	Orphaned <code>llama-server.exe</code> processes eating VRAM. Killing <code>ollama app.exe</code> does not reap them; each restart strands another	<code>Stop-Process -Name "ollama app", "ollama", "llama-server" -Force</code> , then restart. Verify exactly 1 <code>llama-server</code>
Generation slows to ~2.6 chunks/min	Something else is using the GPU	Do not run CUDA work while stage 2 runs
High pinned at hard cap	Splitter cascade found no boundaries	Fix before spending anything on Phase 2
Many rejected pairs	Chunks fragmentary/tabular, or model ignoring the stand-alone rule	Check Phase 1 output; <code>qc.py</code> already drops these
Loss flat from step 0	LoRA not attached, or <code>prepare_model_for_kbit_training</code> skipped	4-bit weights are frozen ints — without it gradients cannot flow through the base
Loss → 0 unrealistically fast	Overfitting, or train/val leakage	Run <code>test_split.py</code>
Training "works" but is 30-60× slow	Model exceeds VRAM. On Windows WDDM pages to system RAM over PCIe instead of raising OOM	Check <code>sec/step</code> and the GPU shared-memory counter, NOT for an OOM. 8B bf16 hit 177 s/step vs 4.74 in 4-bit (§14). Use <code>--qlora</code>
CUDA OOM in the bf16 arm	bf16 + optimizer + activations > VRAM	Drop both arms to a smaller base and record the OOM as a result
Tuned loses to base in stage 5	Data quality , almost always	Audit Phase 2 groundedness before touching hyperparameters
Fluent but fabricated specifics	Expected — LoRA does not install facts	Not a bug. Needs retrieval (§11)

10. Success criteria

Must have:

- ✓ `lora_from_scratch.py` passes equivalence against `peft`
- ✓ Both A/B runs complete; measurements recorded against §5
- ✓ Every §5 prediction confirmed or explicitly falsified with a reason (§13)
- ✓ Tests green

Nice to have:

1. ⚠️ Tuned model beats base on **loss** (\$15, 3.8×); faithfulness still ungraded
2. An 8B QLoRA adapter that demonstrably could not be trained in bf16 here

Criterion 3 is the important one. A falsified prediction is a *successful* outcome — it means the measurement taught something the documentation did not. Only an unexamined result is a failure.

11. Out of scope (candidate v2)

- **Retrieval over `chunks.jsonl`** — the correct fix for fabricated specifics. The extraction work is already shared.
 - **OCR + font remapping for the 8 excluded PDFs** (~1,780 pages) — would restore the entire Lal Kitab tradition.
 - **Continued pretraining** on raw corpus text. Rejected for v1: ~3M tokens is thin for CPT, and it degrades instruction-following that must then be recovered.
 - **Claude Batch generation** (~\$22-55) for a higher-quality teacher, or a side-by-side against the local `qwen3:14b` data. `02_generate.py` is ready and shares the prompt.
 - **Serving, UI, end-user evaluation.** This is where SDD *would* pay for itself.
-

12. Progress log

Date	Event
08-26	Corpus surveyed: 52 PDFs, 44 usable, 8 excluded
08-26	Stage 1 complete — 1,904 chunks; two extraction bugs fixed
08-26	Teacher benchmarked: <code>qwen2.5:7b</code> unusable (0 pairs), <code>qwen3:14b</code> chosen
08-26	<code>OLLAMA_NUM_PARALLEL</code> tuning attempted and reverted — it hurt
08-26	Blackwell stack validated: sm_120 kernels, NF4 forward, LoRA backward
08-26	Phase 0 complete — 9/9, matches <code>peft</code> ; <code>inject_lora</code> freeze bug caught
08-26	Orphaned <code>llama-server</code> processes found and killed — 0.5 → 4.3 chunks/min
08-26	Stage 2 launched (detached, resumable); <code>04_train_hf.py</code> + <code>06_compare.py</code> written
08-26	Stage 2 finished in 456.9 min — 14,517 pairs from 43 sources, 407 QC-rejected
08-26	Split: 12,695 train / 910 val / 912 test, 5 whole books held out
08-26	Smoke test passed; bf16 arm fits (12.22 GB) — the plan's biggest unknown
08-26	Both arms trained at 500 steps; 3/5 predictions held (§13)
08-26	<code>06_compare.py</code> one-sided-threshold bug found and fixed — it had masked a miss
08-26	<code>07_demo.py</code> : adapter on/off shows style transfer worked, facts did not
08-26	Phase 6 — 8B bf16 paged 11 GB to RAM (177 s/step); 4-bit ran at 4.74 s/step
08-26	17 more tests written; 26 passing
08-26	§15 validation: adapter beats base 3.8× on held-out perplexity

13. Results (2026-08-26)

Qwen3-4B-Instruct · 12,695 training pairs · 500 steps · effective batch 16 · $r=32$ · all 7 linear modules · `assistant_only_loss` · RTX 5070 Ti. **Only the base quantization differed between arms.**

	LoRA bf16	QLoRA nf4	Δ
Peak VRAM	12.22 GB	5.54 GB	−54.7%
Base weights	8.04 GB	2.72 GB	−66%
Sec / step	2.882	3.300	+14.5%
Wall (500 steps)	24.0 min	27.5 min	+3.5 min
Train loss	1.4892	1.5039	+0.0147
Eval loss	1.4622	1.4729	+0.0107
Trainable params	66,060,288	66,060,288	0
Adapter on disk	264.3 MB	132.2 MB	−50%

Scorecard: 3 / 5 held

Prediction	Result	Verdict
VRAM saving 50–60%	54.7%	✓ HOLDS
Speed cost 25–40%	14.5%	✗ MISSED (below band)
Eval loss $\Delta < 0.15$	0.0107	✓ HOLDS
Adapter size identical	50% different	✗ MISSED (above band)
Trainable params identical	0 diff	✓ HOLDS

Miss 1 — the speed penalty was half what I predicted

Predicted 25–40% slower; measured 14.5%.

This workload is partly memory-bandwidth-bound, and 4-bit weights are a quarter the bytes to move. That bandwidth saving offsets much of the dequantization cost. On Blackwell with a bf16 compute dtype the dequant path is also cheap. The conventional "30–40% slower" figure comes from older hardware where the offset is weaker.

A tooling bug hid this initially. `06_compare.py` originally checked a one-sided ceiling ($\leq 60\%$), so 14.5% reported HOLDS despite missing the stated band by half. The checker now enforces two-sided bands. A *prediction that is wrong in a direction you like is still wrong* — and a scorecard lenient enough to pass it is not measuring anything.

Miss 2 — adapter sizes differ, but not meaningfully

264.3 MB vs 132.2 MB, an exact 2:1 ratio. Inspecting the files:

runs/lora-bf16	504 tensors	dtype=F32	4.00 bytes/param
runs/qlora-nf4	504 tensors	dtype=BF16	2.00 bytes/param

Same 504 tensors, same 66,060,288 parameters — **different storage dtype only**. peft creates adapter weights in fp32 by default; in the QLoRA path `prepare_model_for_kbit_training` leaves them in the bf16 compute dtype.

The adapters are mathematically equivalent. **The trap:** anyone comparing adapter file sizes would conclude "QLoRA produces smaller adapters." It does not. This is precisely what a control variable exists to catch.

A related artifact: `trainable %` reads 1.616% vs 2.908% only because the *denominator* differs — 4-bit weights pack two per byte, so the base counts as 2.27B instead of 4.09B. The numerator is identical in both runs.

Conclusion

QLoRA on this hardware is close to free: half the memory, ~15% more time, no measurable quality cost. The 6.7 GB it frees is the difference between a 4B and an 8B model — and an 8B in bf16 (~16 GB of weights) does not fit on a 15.9 GB card, while in 4-bit it does. That is the practical case for QLoRA, now measured rather than assumed.

The honest caveat: 500 steps is ~0.63 epochs, so both adapters are under-trained. That does not affect the *comparison* — both arms saw identical data for identical steps — but the adapters themselves are not finished models.

14. Phase 6 — 8B on a 16 GB card (2026-08-26)

The 4B comparison showed QLoRA as a **trade**: half the memory for ~15% more time. At 8B it stops being a trade.

	8B bf16	8B QLoRA nf4
VRAM used	15,864 MiB (132 MiB free)	11,520 MiB (4,018 free)
Base weights	~16 GB (does not fit)	6.17 GB
Spilled to system RAM	~11 GB	0
sec / step	177.7	4.74
100 steps	4.9 hours	7.9 min
Eval loss	— (abandoned)	1.4250

QLoRA is 37× faster on the same model, hardware, and data.

The prediction was wrong, and the way it was wrong matters

\$5 and \$9 both assumed 8B bf16 would OOM. It did not. On Windows, WDDM lets the GPU oversubscribe VRAM by paging to system RAM over PCIe. The model loaded, reported success, and trained — at 177 s/step with ~11 GB crossing the bus every step. Confirmed via performance counters:

6.56 GB + 4.35 GB + 0.21 GB spilled to system RAM
GPU pinned at 100%, sustained 177 s/step across 4 steps

This is a worse failure mode than an OOM. An OOM is immediate and unambiguous. This looks healthy — progress bar advancing, GPU at 100%, loss computing — and is only detectable by noticing that a 500-step run reports an ETA of 24 hours. Anyone benchmarking without watching step time would conclude "8B trains fine in bf16 on 16 GB."

Do not use OOM as your fit test on Windows. Use step time and the spill counter.

What this establishes

Without 4-bit, 8B on this card is technically runnable and practically useless. With it, the base drops to 6.17 GB, fits with 4 GB spare, and trains at a normal rate. That is QLoRA's actual purpose — not saving memory on a model that already fits, but making a model trainable that otherwise is not.

Two side observations: the 8B adapter reached a better eval loss (1.4250) than the 4B (1.4729) in one-fifth the steps — a bigger base adapts faster. And trainable_params rose to 87.3M from 66.1M, as expected at r=32: more layers, wider dimensions.

15. Validation — adapter vs untuned base (2026-08-26)

§13 compared the two arms to each other. It never compared either to doing nothing. This section closes that gap.

Method: pipeline/08_validate.py . Both conditions use the same loaded weights via peft's disable_adapter() , so nothing but the adapter can differ. Examples come from test.jsonl — 5 books held out of training entirely, with test_split_has_no_book_leakage confirming zero overlap.

Held-out loss (n = 250)

	BASE	TUNED	
Assistant-token NLL	2.7704	1.4463	−1.3241
Perplexity	15.96	4.25	3.8× better

Loss on assistant tokens only, masked identically to training. The fine-tuning unambiguously worked — on books it has never seen, the adapter more than halves the loss.

Style metrics (n = 20 generated answers)

Mechanical regex probes — countable, no judge model, no API key.

	BASE	TUNED	
Doctrinal framing ("states", "holds")	25%	50%	✓ better
Hedging preamble	40%	5%	✓ better
Markdown headers	65%	0%	✓ better
Personal prediction	0%	0%	—
Names the source text	75%	70%	≈ noise at n=20
Uses Sanskrit terminology	60%	30%	✗ worse
Mean answer length	119 words	73 words	—

Format training succeeded completely. Markdown 65% → 0%, hedging 40% → 5%, length nearly halved. The base model's rambling essay register was replaced by terse doctrinal answers — exactly what 12,695 terse examples should produce.

Unexplained: Sanskrit terminology fell

Predicted to rise; it halved. Working hypothesis: the base rambles for 119 words and sprays terms (*Shani*, *Kendra*, *Navamsa*) as padding, while the adapter answers in 73 words and uses a term only where one is needed — fewer words, fewer terms. **This is a hypothesis, not a finding.** It is equally possible the adapter genuinely lost vocabulary. Recorded as open rather than explained away.

What remains unmeasured: factual accuracy

Low held-out loss means the adapter **predicts corpus-style text** far better. It does **not** mean its facts are right. `07_demo.py` showed it answering "the 2nd, 5th, 7th, 8th, 10th, or 11th house lords" where Phaladeepika says "the ascendant lord, 7th lord, 5th lord, Jupiter, the planet aspecting the 5th, the planet occupying the 5th." Fluent, correctly shaped, confidently wrong.

Loss and factual correctness are different quantities, and only the first is demonstrated here. Closing this needs `05_eval.py judge` — 150 held-out questions graded for support and fabrication against their source passages — which requires an API key.

Verdict

Claim	Status
QLoRA ≈ LoRA in quality	✓ measured (§13)
QLoRA saves 54.7% VRAM	✓ measured (§13)
8B: 37× faster in 4-bit	✓ measured (§14)
Adapter beats the untuned base	✓ 3.8× perplexity, held-out
Format/register transferred	✓ markdown 65%→0%, hedging 40%→5%
Adapter is factually reliable	✗ not measured — demo suggests it is not

The adapter performs well on everything the training was designed to install. The factual limitation predicted in §1 is real, and its magnitude is still unquantified.

Appendix — file map

Path	Status
pipeline/config.py	✓ shared paths, skip list, generation prompt + schema
pipeline/qc.py	✓ question sanitizer + structural gate
pipeline/01_extract.py	✓ PDFs → chunks
pipeline/02_generate_local.py	✓ Ollama generator (resumable) — in use
pipeline/02_generate.py	✓ Claude Batch generator (alternative)
pipeline/03_split.py	✓ split by book
pipeline/04_train_hf.py	✓ transparent peft + bitsandbytes trainer
pipeline/04_train.py	⚠ Unsloth variant — blocked, triton missing
pipeline/05_eval.py	✓ A/B generation + faithfulness judge
pipeline/06_compare.py	✓ prediction scorecard
pipeline/check_env.py	✓ Blackwell stack gate
pipeline/lora_from_scratch.py	✓ the mechanism, ~60 lines
pipeline/tests/test_lora_module.py	✓ 9/9
tools/md2pdf.py	✓ this document → PDF
tools/watch.py	✓ live GPU + job monitor
runs/lora-bf16/	✓ bf16 adapter + metrics
runs/qlora-nf4/	✓ 4-bit adapter + metrics
runs/8b-qlora/	✓ 8B 4-bit adapter (§14)
pipeline/07_demo.py	✓ adapter on/off side-by-side
pipeline/tests/test_eval_unswap.py	✓ 5/5
pipeline/tests/test_data_pipeline.py	✓ 12/12
pipeline/08_validate.py	✓ base-vs-adapter validation (§15)
runs/qlora-nf4/validation.json	✓ validation metrics

Remaining optional work: `05_eval.py judge` for a faithfulness A/B against the untuned base (needs an API key); the four outstanding tests in §7; and the 36 chunks (1.9%) that produced no pairs, recoverable by re-running the resumable generator.